

Engineering Ethics Issue in the Therac-25 Accidents

Joe Le — Computer Engineering

Introduction of the Case

The Therac-25 was a computer-controlled radiation therapy machine used across the US and Canada in the 1980s. Between 1985 and 1987 it was involved in six known accidents, each one exposing a patient to a massive radiation overdose — injuries and deaths resulted. When Nancy Leveson and Clark Turner investigated, what they found wasn't a single bad line of code. It was a stack of failures: weak hazard analysis, thin testing and QA, too much confidence in the software, operator feedback that was more confusing than helpful, and a slow, dismissive response to the first reports that something was wrong. Compared to earlier models, the Therac-25 leaned harder on software and dropped some of the independent hardware interlocks that used to catch these problems. Under the right (wrong) sequence of fast operator input, the machine could land in a race condition and an unsafe state — and when it did, operators often just saw a cryptic error code instead of any real warning that a patient might be about to receive a lethal dose.

Engineering Ethics Issue

The ethics issue I want to focus on here is simple to state and hard to actually live by: public safety has to come before confidence in your own design, and before the pressure to keep a machine running. IEEE Code of Ethics I.1 is explicit about this — engineers are supposed to hold public safety, health, and welfare paramount, and to disclose anything that could endanger the public. Therac-25 violated that in practice, because when reports of serious malfunction came in, the initial response was to explain them away rather than shut the machine down and actually investigate. It also runs headlong into Code I.5, which calls on engineers to accept honest criticism, own their mistakes, and be realistic about what the evidence actually supports. “The software can't fail” is not a safety argument, full stop — not in a system where failure means a lethal radiation dose. The only honest starting assumption is that hardware, software, operators, sensors, and interfaces can all fail, and the job of the engineer is to make sure no single one of those failures is enough to cause a disaster on its own.

Mitigation

Roughly, the causes split into design problems, verification problems, and organizational problems. On the design side, safety leaned too much on software and not enough on independent hardware interlocks. The software itself had race conditions and let the machine slip into unsafe state transitions. Testing never really pushed on realistic operator behavior — fast typing, odd timing, weird combinations of hardware and software state. And the hazard analysis and incident-reporting process was thin enough that early complaints didn't trigger the system-wide investigation they should have. A better plan starts before a line of code is written: do a formal hazard analysis first, and identify which hazards have to be prevented independently of the control software, not by it. Safety-critical outputs should default to a safe state. Independent interlocks should physically block dangerous energy delivery — not rely on a flag being checked correctly. Requirements need explicit fault states, timeouts,

plausibility checks, sensor-disagreement detection, and alarms operators can actually understand. Verification needs unit tests, integration tests, fault injection, boundary tests, rapid-input sequences, hardware-in-the-loop testing, and acceptance criteria you can trace back to a requirement. And every serious field incident should trigger immediate containment — preserve the logs, find the root cause, tell the users, and revalidate before the system goes back into service.

That's directly shaped how I'm approaching OSBAMS. A battery test platform is also controlling hazardous stored energy, just at a smaller scale. So OSBAMS gets a hardwired E-stop path, a contactor that's normally off, fuse protection, firmware fault latching, a watchdog, redundant voltage checking, temperature monitoring, and staged validation before a real battery ever gets connected. The lesson I keep coming back to: software can supervise safety, but it can't be the only thing standing between the system and a hazard.

Lessons and Conclusions

The biggest thing Therac-25 shows is that software reliability and system safety aren't the same thing — a system can sail through every normal demo and still be hiding a dangerous combination of states underneath. Engineers need to treat weird failures, operator complaints, and measurements that don't add up as things worth investigating, not annoyances to wave away. Any safety claim needs test evidence, independent protection, and honest assumptions about how things can fail behind it. For my own work, the lesson I'm taking most seriously is to build safety into the architecture from day one and to just stop testing the moment the system lands in a state I can't explain. In OSBAMS terms, that means validating every sensor and safety path on its own, keeping the battery disconnected while any bring-up fault is unresolved, and requiring a known-good IDLE state before moving to higher-energy testing. Engineering ethics isn't some separate box you check after the design is done — it shapes the architecture, the validation plan, the documentation, and what you do the moment something behaves unexpectedly.

References

- [1] Leveson, N. G., & Turner, C. S. (1993). An Investigation of the Therac-25 Accidents. IEEE Computer, 26(7), 18–41. https://www.cse.msu.edu/~cse470/Public/Handouts/Therac/Therac_1.html
- [2] IEEE. Code of Ethics (revisions adopted June 2020). <https://ieee-cas.org/about/ieee-code-ethics>
- [3] Ethics Unwrapped, The University of Texas at Austin. Therac-25 case study. <https://ethicsunwrapped.utexas.edu/case-study/therac-25>